

クロコ プロジェクト仕様書（統合版）

本書の位置づけ

このフォルダにあった以下2点の内容を、**新しい判断を加えず**に整理・統合したものです。

- クロコ第一段階_実装仕様書.md（対話で確定させてきた「第一段階」の確定/未決定事項）
- Claude_Code運用計画（クロコ統合版）.pdf（3層アーキテクチャによる将来構想・気づき口グ）

2つの文書は**時間軸が異なる**（PDFはラズパイ等を含む将来の統合構想、mdファイルは「今回はラズパイ等を使わない」と明言した第一段階の合意）。

本書はこの違いを混同しないよう、「**第一段階（確定）**」と「**将来構想（未着手）**」を明確に分離して記載する。

未決定事項について本書が推測で埋めた箇所は一切ない。

1. 目的

- 一言で：認知的コスト・起動コストを下げるための、Notion＋クロコによる個人用ワークフロー自動化。
- PDFより：認知的コストと実行性の乖離による生産コストの低下を補う装置。原動力は2E／ギフテッドと呼ばれる特性のある人のためだが、組織への応用可能性もある。

2. 第一段階のスコープ（確定事項）

1. **範囲**：クロコ＋Notionのみで構築する。ラズパイ／Dify／Cloudflare Tunnel／Syncthingは今回は使わない（将来の拡張候補として温存、詳細は3章）。
2. **Notion AIを契約する**（Businessプラン、\$20/user/月）。
3. **Notion AIの位置づけ**：最終システムはNotion AIに依存させない。最終形は「クロコ＋Gemini API」で完結させる方針。Notion AIは契約から自己推薦書（大学出願書類）が書き終わるまでの、入口の便利機能という位置づけ。
4. **環境制約**：PCは1台のみ、常時起動サーバーは存在しない。ラズパイは未導入。
→ Webhookによるリアルタイムトリガーはこの環境では意味を持たないため保留。
5. **PCの運用スタイル**：使うたびにPCを再起動する。
→ 常駐タスクや個別登録プログラムの数を増やしたくないという強い選好がある。
→ 起動時トリガー（PC起動/ログイン時にまとめて処理を走らせる）方式で構わない。
6. **起動方法の方向性**：Notion取得・整形・クロコ起動を別プログラムとして個別登録せず、**1本のコマンド（パイプでつなぐ）**にまとめる方向。

※ 具体的なスクリプト名・CLIオプション・プロンプト文言などの実装細部は未決定。

7. 認証情報（APIキー等）の保管方法：.env ファイルを、本プロジェクトフォルダ（Desktop配下）の外（例：%USERPROFILE%\croco\.env）に置く方針。

- 理由：このフォルダは現状OneDrive同期対象外だが、将来設定が変わった場合や、フォルダを人に渡す場合に備え、平文の秘密情報をプロジェクトフォルダ内に置かないため。
- 1台の個人PC・多人数アクセスなしという環境を踏まえ、Windows資格情報マネージャー等への格上げは将来（サーバー常時公開等の段階）に見送る。
- 具体的な配置パス・読み込み方法は実装時に決定。

2.5 第一段階のワークフロー（対話で追加確定した内容）

2章の確定事項をふまえ、以下のワークフローを合意した。

1. **入力**：スマホでブレストしたアイデア。
2. **起動トリガー**：確定。PC起動時にクロコ（Claude Code）を立ち上げる方式でいく。定期実行（起動トリガーとは別に、一定間隔でも走らせるか）は**不要と確定**。理由：本人は作業の区切りなどでPCを頻繁に再起動する習慣があるため、起動時トリガーだけで実質的に十分な頻度になる。
 - 判断材料：このPC（メモリ8GB）で「YouTube動画を見ながら裏で実装させる」のは画質を落とせば行けそう、「重いゲーム（スタレ等）と同時」は厳しそう、という実地感覚から、**動画を見ながら裏で実装が進む運用を前提**にすることにした。
 - よって「趣味でPCを一瞬開いただけの時に邪魔」という懸念よりも、「起動したらそのまま実装が進む」ことを優先する判断。
3. **Gemini APIの役割**：確定（当初の「要約」から修正）
 - スマホから送られたブレストの生ログを、**話題の切れ目で分割するだけ**。本文は要約・言い換えせず**逐語転記**、タイトルは本文を指す短いラベルのみ（意味理解・仕様化はしない）。
 - 理由：Geminiが要約した時点で失われた情報はクロコ側で復元できない。非可逆圧縮を、何を作るか決まっていない段階で挟むのは危険なため。
 - 分割・ラベリングした結果をNotionのDBに登録する（1セッション＝1回のAPI呼び出しで、JSON配列として複数アイテムをまとめて返させる）。
 - モデル**：確定＝**Gemini 3.5 Flash**。要約・分類・抽出用途ではFlash系がコスパ・性能とも妥当という調査結果（2026年7月時点）を踏まえて選定。本人としてはGemini自体の性能に一般的な不満（本質を突けない）があるが、「分割・転記のみで解釈はさせない」設計にしたことで、この用途ならFlashで十分という判断。実運用で精度不足を感じたら3.1 Proへの切り替えを検討。
 - クロコ側のトークン消費への影響**：問題なしと確認。逐語転記でも1アイデアあたり数百～数千トークン程度で、Claude Codeのコンテキスト規模からすれば無視できる量。

またクロコは着手するアイテム単体（+自分の進捗ログ）だけを読めばよく、DB全体を毎回読み込む設計ではないため、要約して圧縮する必要性自体が薄い。

- 唯一気に留めるべきは、**実行結果（進捗ログ）が同一アイテムを何セッションもまたいで続けた場合に際限なく伸びること**。ただし現状の利用規模では大きな問題になるとは考えにくく、実装時に様子を見て必要なら対処する程度でよい。

- （プロンプト文言の具体的な中身・スキーマ設計は実装時に決定。設計方針の詳細は3章「参考資料」参照）

4. **Notion Inbox DBの構造**：本人からクロコ（実装者）に設計を一任された。プログラムで扱いやすい形であれば形式は問わない、という条件のみ。

- 実装時にクロコが提案する形で確定させる（本書では具体的なプロパティを先出ししない）。

5. **スコープの境界：更新**。当初は「パイプの担当はInbox DBへの要約登録までで、実際の実装作業はClaude Codeを普段どおり使うだけ（パイプの範囲外）」としていたが、**この整理は撤回**。

- このプロジェクトの目的はそもそも実装を進めることなので、**起動時にクロコが立ち上がったなら、そのままアイデアの実装に自動で着手する**、という前提に変更。
- つまり「起動→（Inbox登録済みの）アイデアを拾ってクロコが実装を始める→本人は動画等を見ながらそれを横目に見ている」という、無人（本人が張り付いていない）実行が前提になる。
- この変更により、**クロコの実行時オプション（許可ツール・パーミッションモード等、4章）の重要度が上がった**：本人が横で承認operationをするわけではないため、何らかの形で無人実行できる権限設定が必要になる。ここはまだ未決定。

6. **予定（スケジュール）のアプライ：実装方式が確定**。ブレスト中に日程が決まった場合、それをスマホから見えるようにしたい、という要望から出発。

- 将来的にはDify運用開始後にそちらへ寄せる想定だが、それまでの間はNotionカレンダーで実装する方向は維持。
- **仕組み**：Notionには「別の場所にpushする」という概念はなく、カレンダービューは同じDBの上に乗る表示形式の一つ。よって**別DBは作らず、同じInbox DBのままでよい**。
 - Inbox DBに「カレンダービュー」を1つ追加し、フィルタで**種別＝予定のみ表示**
 - カレンダー表示に使う日付は、処理ログ用の各種日時とは別に新設した「**予定日**」プロパティ（10.のDB構造参照）
- これにより「Inbox DBとの関係」の論点は解消。予定アイテムは同じDB内で種別＋予定日により区別・表示される。

7. **無人実行で扱う想定タスクの性質**：本人の想定用途は主に

- Dify関連の日常用アプリ制作
- 受験関係のタスク
- 個人的に受注した小規模プロジェクト（マイコンでのデータロガー等の簡単なもの）であり、「**大きな事故さえ起きなければ強い権限は不要**」という本人の判断。パーミッ

ションモードの検討はこの前提（＝重大事故の防止が最優先で、日常的な細かい承認は求めない）で進める（4章参照）。

8. **AI利用が問題になりうる文書の扱い**：自己推薦書のような、AI作成だと問題になり得る文書は**現時点ではAIに作成させる予定がなく、無人実行の対象外**。

- ただし将来、大学のレポート等でAIを使う可能性はあるとのこと。その際の安全策として、GitHubやNotionでバージョン管理し、そこへのpushという形にする案が本人から出ている（まだ検討段階、決定ではない）。

9. **パーミッションモード：確定**。7・8を前提に、「日常操作は自動承認、真に危険な操作（`git push --force`／共有ブランチへの `git reset --hard`／プロジェクト外への書き込み・削除／`rm -rf` 的な広範囲削除／外部送信系操作）だけ明示的に拒否する」方針で進める。

- 拒否リストの具体的なパターン一覧（`settings.json`の記述等）は実装時に詰める。

10. **Notion Inbox DBの構造**：たたき台を本人が提示、方向性は良好

	プロパティ名	型	値/選択肢	書き込み主体
	タイトル	title	自由記述	Gemini API（自動生成）
	（ページ本文）	page content	自由記述	Gemini API
	種別	select	アイデア / 予定	Gemini API（作成時）
	予定日	date	-	Gemini API（種別＝予定の場合）
	ステータス	select	未処理 / 処理中 / 完了 / 要確認	作成時=Gemini（常に未処理固定）、以降=クロコ
	発話日時	date	-	将来：Gemini API（スクリプト側で正確に記録）
	作成日時	created_time	-	自動
	処理開始日時	date	-	クロコ（最初に未処理→処理中にした時）
	処理日時（完了）	date	-	クロコ
	実行結果	rich text	自由記述（進捗ログ）	クロコ

- 「失敗時は未処理のまま残す→次回起動時に自然に再試行」という設計により、**冪等性・エラーハンドリングの土台は解決済み**（4章の未決定事項から実質的に前進）。
- 「**処理中**」ステータスの位置づけ：**確定**。クラッシュ時の特別扱い（未処理に戻す等）ではなく、そもそも「**処理中**」＝複数回の起動セッションをまたいで実装が進行中の状態と

いう位置づけ。実装は1回の起動＝1ターンとは限らず、対応（ターン）ごとに状況に応じて動的にステータスが更新される想定（1回の対応で1回更新、という粒度は本人の想定であり、実装時に検証）。

- よって次回起動時は、**新規の「未処理」より「処理中」を優先して処理を再開する**という優先順位にする。これにより、通常の再開処理と、実行が中断された場合の復旧処理が同じ仕組みで自然に統合される（別途クラッシュ復旧ロジックを設ける必要がない）。
- **進捗の引き継ぎ方法：確定**。実行結果（rich text）に、gitのコミットメッセージ的な簡潔な進捗ログを追記していく形式とする。次セッションはこれを読んでフックとして続きを実装する。追記の書式・粒度（1ターンごとか、まとめてか等）は実装時に決定。
- **種別（アイデア/予定）プロパティ：確定**。同一Inbox DB内に混在させ、種別プロパティで区別する方針とした（別DBには分けない）。
- **研究解析用のログ：方針確定（詳細は未定）**。本人より「後でこのデータを解析して研究用プロジェクトの参考にするかもしれない」という動機が示された（4章「評価指標・ログの取り方」と同根の話として統合）。
 - 現状のハードルは高くなく、「今はちゃんとログさえ取れていればいい」（詳細な評価指標の設計は後回しでよい）という前提。
 - この前提のもと、上表に**発話日時・処理開始日時を追加提案**した。これにより後から「発話→着手までのラグ」「着手→完了までの処理時間」「何ターンかかったか（実行結果ログの件数）」「種別ごとの傾向」「最終ステータスの分布」といった分析ができるようになる、という設計上の狙い。
 - ただし発話日時が実際に取得できるかは、スマホからの入力手段自体が未定なので未確定。
- 残る論点（未確定のまま）：
 - 作成日時と発話日時のズレの扱い（発話日時が取得できない場合の代替を含む）

11. Gemini APIプロンプト設計：方向性を確定（文言そのものは実装時）

- **呼び出し単位**：プレスト1セッション＝API1回。生ログをまとめて渡し、JSON配列で複数アイテムを一括で返させる（アイテムごとに呼ばない）。
- **出力方式**：`responseMimeType: application/json + responseSchema`（構造化出力）。スキーマはfield単位で `description` を付け、プロンプト本文にスキーマを重複記載しない（重複は出力品質を落とすとGoogleが明記）。種別は自由文字列ではなく `enum`（"アイデア"/"予定"）にして揺れを防ぐ。
- **スキーマに含めないもの**：ステータス（常に"未処理"固定なのでスクリプト側でハードコード）、発話日時・処理日時・実行結果（クロコ側が書くものなので対象外）。
- **現在日時はスクリプトから注入**：Notion AIの件で確認済みの通りLLMは現在時刻を知らないため、「来週の火曜」等の相対表現を予定日に変換するには必須。この注入日時は将来「発話日時」カラムにもそのまま使える。

- **thinking**：3.5 Flashでは thinking_budget ではなく thinking_level。分割・転記に深い推論は不要なため minimal から開始し、精度が問題になれば上げる。
- **temperature**：低め（0～0.3）。抽出タスクなので創造性は不要。
- これらはコスト最適化目的ではない（このボリュームではAPI従量課金・レート制限は無風であることを確認済み）。目的は精度と、クロコが後で読む際の情報保存のため。

12. **無人実行の安全対策：追加で確定**。9.（自動承認＋危険操作の拒否）に加え、実装時に以下も行う。

- **Notion連携トークンのスコープをInbox DBのみに絞る**：ワークスペース全体ではなく、Notion Integrationの共有（Connect）をInbox DBだけに限定する。鍵の漏洩・バグ時の被害範囲をこのDBだけに閉じるため。
- **プロジェクトごとにgitブランチ／worktreeを切って作業させる**：無人実行中の失敗がmainに波及しないよう、被害範囲をブランチ単位で最初から区切っておく。「危険操作の拒否」が行為ベースの防御なのに対し、こちらは範囲ベースの防御として別レイヤーで効く。
- **同一アイテムの再試行に上限を設ける**：一定回数（実装時に具体値を決定、たたき台は3回）進捗がないまま繰り返したら、自動的に「要確認」に落として人間待ちにする。無限ループの防止。
- **止め方の確認**：タスクマネージャーやターミナルを閉じる等、放置中に異常に気づいた際すぐ止められる手段を本人側で把握しておく（実装不要、運用上の確認事項）。
- これらはこのクロコ・パイプライン自体を実装するタイミングで適用する対策であり、今の時点（仕様検討中）で何かを設定するものではない。

3. 将来構想（PDFより／現時点では未着手・第一段階には含まない）

⚠ 以下はPDF「クロコ統合版」に記載された将来構想であり、2章の「第一段階の確定事項」とは矛盾する（ラズパイ等を使う前提）。今回は着手しない範囲として記録のみ残す。

なお、PDFの「気づきログ」にあった起動コスト／インタラクション負債に関する考察は、本人ではなく別の人物についての話であり、本人のためだけの第一段階には不要な要素として本書では割愛した。

参考資料：ブレスト用対話AIアーキテクチャ（第二段階向けの調査メモ）

第二段階（スマホからの対話ブレスト部分）を作る際の参考として、一般的な会話AIアーキテクチャを調査済み（2026年7月時点）。

- 音声を使う場合の基本形：**STT（音声→テキスト）→ LLM → TTS（テキスト→音声）**のパイプライン。
- より新しい方式：STT/TTSを分けず、音声を直接入出力できる**統合Audio LLM**を使う構成もあり、レイテンシが小さい。

- **Function calling**：LLMが会話の中で「これはメモとして保存すべきか」等を自分で判断し、ツール（DB書き込み等）を呼び出す設計。
- **ハイブリッド型**：明確な意図は決め打ちのハンドラで処理し、曖昧なものだけLLMに投げることで、コストと精度を両立させる設計。

まだ着手時期ではないため、これは実装方針ではなく参考メモとして残すのみ。

3層アーキテクチャ

1. スマホ（指示・閲覧の窓口）

- 役割：すべての指示入力の入口。外出先からも操作。
- 主なツール：Dify (PWA)
- 閲覧：予定はNotion（カレンダー画面）で確認

2. ラズパイ（常時起動ハブ：日常タスク+ログ蓄積）

- 役割：日常タスク（例：PDFリンク等からのスケジュール抽出→登録）を自動処理／プロジェクト開発用のプレストログ・仕様書（Markdown）を蓄積・同期
- 主なツール：Dify (OSS)、Gemini API、Docker / Docker Compose、Cloudflare Tunnel（外部から安全にアクセス）、Syncthing（PCへ暗号化同期）
- 方針（クロコ3準拠）：メモリ節約のため、**ラズパイ側でのClaude Code運用は除外**する。

3. PC（本丸開発：Claude Code実行環境）

- 役割：ラズパイから同期された指示書（.md）を最優先コンテキストとして読み込み、環境構築が必要な開発や複雑な実装を進める。
- 主なツール：Claude Code（Anthropic公式CLI）、Node.js / npm

データの流れ（ワークフロー）

A. 日常タスク（スケジュール管理など）

1. スマホ（Dify）：「このPDF（URL）の予定をカレンダーに反映して」等を指示
2. ラズパイ（Dify + Gemini）：URL/PDF等を解析し、予定データを構造化
3. ラズパイ→Notion：Notion API経由でNotionカレンダーに予定を自動登録
4. 結果：スマホで常に最新の予定を確認でき、ノートPCを開く頻度を減らす

B. 本格開発プロジェクト（仕様書→実装）

1. スマホ（Dify）：移動中にアイデア出し・壁打ち（プレストログ生成）
2. ラズパイ（Dify + Gemini）：セッションを要約し、Markdownの「指示書/仕様書」としてローカル保存
3. 同期（Syncthing）：ラズパイの出力フォルダをPCへ暗号化自動同期
4. PC（Claude Code）：同期された.mdを最優先コンテキストとして読み込み、実装開始

フォルダ設計（ログ/成果物の分離、SyncthingでPCにも同期する前提の例）

```
workstation/  
  history/    ... スマホ→Dify→ラズパイで生成されたブレスト/仕様書ログ  
    project-1_fix-notes.md  
    project-2_idea.md  
  output/     ... PC側（Claude Code等）で作った成果物・実行結果  
    project-3_schedule.md
```

狙い：スマホからも成果物を読める（outputを同期しておく）／ブレストログ（history）と成果物（output）が混ざらず再利用しやすい。

セキュリティ/運用上の要点

- 外部アクセス：Cloudflare Tunnelを使い、ポート開放なしで安全にスマホ→ラズパイへアクセス。
- 同期：SyncthingでP2P暗号化同期（ラズパイ→PC）。
- エラー/制限への対処：APIのレート制限やエラー種別を正しく判定し、待機→再試行などのリトライ戦略を入れる（無駄な再試行はエラー種別で分岐して避ける）。

構築ロードマップ（実施手順・PDF記載の将来案）

1. 基盤：ラズパイにRaspberry Pi OS 64bitを導入し、Docker環境を構築
2. ハブ：Docker上にDifyをデプロイ（メモリ消費を最適化）
3. 通信：Cloudflare Tunnelをセットアップし、外部からのHTTPSアクセスを確立
4. 同期：ラズパイとPCにSyncthingを導入し、workstation/を同期
5. 日常タスク自動化：GeminiでPDF等を解析→Notionに予定登録するワークフローをDifyで構築
6. 開発環境：PC側にNode.jsとClaude Codeを導入し、同期された.mdをトリガーに運用テスト

期待メリット（統合）

- PCを開くハードルが下がる：日常の雑務はラズパイが処理し、PCは開発に集中できる
- 安全性の担保：Cloudflare Tunnelにより自宅サーバー公開リスクを抑える
- 開発の再現性が上がる：ブレスト→仕様書（.md）→Claude Code実装、の流れが固定化される

4. まだ曖昧・未決定（他のAIが推測で埋めないこと）

- Gemini APIのプロンプト文言そのもの（役割・モデル・スキーマ設計の方向性は2.5章-3・11で確定：分割＋逐語転記、Gemini 3.5 Flash、構造化出力）
- **評価指標・ログの取り方／作成日時と発話日時のズレの扱い（統合済み、2.5章-10参照）**
 - **背景（本人より）**：これらを気にしているのは、後でこのデータを解析して研究用プロジェクトの参考にするかもしれないから、という理由。したがって「今使う分には多少ズレていても困らない」ではなく、**後から解析に耐える精度・形式で残しておく価値がある**、という前提で考えること。
 - **現時点の到達点**：「今はちゃんとログさえ取れていればいい」（詳細な評価指標の設計は後回しでよい）という本人の方針のもと、Inbox DBに発話日時・処理開始日時を追加提案済み（2.5章-10）。**発話日時カラムは今から作るが、当面は空欄で運用し、作成日時で代用する**。値を入れ始めるのはGemini自動化パイプラインができてから、という時期も確定した（2.5章-10）。
 - **残る未確定**：具体的な評価指標そのものの設計（何をどう分析するか）はまだ先の話。
- ラズパイ導入など、3章「将来構想」への移行時期・要否そのもの

5. 進め方についての方針

本人（プロジェクト所有者）は対話を通じて考えをまとめるタイプであるため、まとめて詳細仕様を先出しするのではなく、**1トピックずつ対話で確定させていく**方式を取っている。

また、詳細に書き込みすぎた仕様書は高性能なコーディングAI相手にはかえって柔軟性を奪う場合があるため、実装の細部（具体的なファイル名・プロンプト文言等）は意図的にまだ書き込んでいない。

本仕様書もこの方針に従い、4章の未決定事項を埋めるための新たな提案は含んでいない。